

Arrays: Representation to Sparse Matrices

Address Calculation, Multidimensional Arrays,
Ordered Lists and Sparse Matrix Representation

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Array: Logical and Physical Views

Logical View

An array is a mapping:

Index $\longrightarrow$ Value

Example:

0 $\rightarrow$ 10

1 $\rightarrow$ 20

2 $\rightarrow$ 30

Physical View

The values are normally stored in consecutive memory locations.

1000 $\rightarrow$ 10

1004 $\rightarrow$ 20

1008 $\rightarrow$ 30

Array: Logical and Physical Views

Logical View

An array is a mapping:

Index $\rightarrow$ Value

Example:

0 $\rightarrow$ 10

1 $\rightarrow$ 20

2 $\rightarrow$ 30

Physical View

The values are normally stored in consecutive memory locations.

1000 $\rightarrow$ 10

1004 $\rightarrow$ 20

1008 $\rightarrow$ 30

Important Distinction

The logical array is defined through indices and values. Contiguous memory is its usual

Why Does an Array Need Contiguous Memory?

Suppose:

```
int A[5] = {10,20,30,40,50};
```

Base address = 1000

Size of each integer = 4 bytes

1000	1004	1008	1012	1016
10	20	30	40	50
A[0]	A[1]	A[2]	A[3]	A[4]

Why Does an Array Need Contiguous Memory?

Suppose:

```
int A[5] = {10,20,30,40,50};
```

Base address = 1000

Size of each integer = 4 bytes

1000	1004	1008	1012	1016
10	20	30	40	50
A[0]	A[1]	A[2]	A[3]	A[4]

The address can be calculated directly without visiting earlier elements.

Random access = $O(1)$

Base Address and Offset

The array name represents the base address:

$$BA(A) = \text{Address of the first element}$$

For zero-based indexing:

$$\text{Offset of } A[k] = k \times w$$

Therefore:

$$LOC(A[k]) = BA(A) + k \times w$$

where:

- $BA(A)$ is the base address.
- k is the index.
- w is the size of one element.

General 1D Address Formula

When the lower bound is not necessarily zero:

$$LOC(A[k]) = BA(A) + w(k - LB)$$

where:

- LB is the lower bound of the array.
- $k - LB$ gives the number of elements skipped.

The length of an array is:

$$\text{Length} = UB - LB + 1$$

where UB is the upper bound.

Numerical Example: One-Dimensional Array

Given:

$$A[-3 \dots 8]$$

$$BA(A) = 2000$$

$$w = 4 \text{ bytes}$$

Find the address of $A[5]$.

Numerical Example: One-Dimensional Array

Given:

$$A[-3 \dots 8]$$

$$BA(A) = 2000$$

$$w = 4 \text{ bytes}$$

Find the address of $A[5]$.

$$LOC(A[5]) = 2000 + 4(5 - (-3))$$

$$= 2000 + 4(8)$$

$$LOC(A[5]) = 2032$$

Interactive Check

Given:

$$B[4 \dots 12]$$

$$BA(B) = 500, \quad w = 2$$

Find:

$$LOC(B[9])$$

- ① 510
- ② 518
- ③ 526
- ④ 528

Interactive Check

Given:

$$B[4 \dots 12]$$

$$BA(B) = 500, \quad w = 2$$

Find:

$$LOC(B[9])$$

- ① 510
- ② 518
- ③ 526
- ④ 528

$$LOC(B[9]) = 500 + 2(9 - 4)$$

Array Representation: Deeper Observation

The address formula explains two important properties.

Property 1: Fast Access

Only arithmetic operations are required.

$$A[k] \Rightarrow O(1)$$

Array Representation: Deeper Observation

The address formula explains two important properties.

Property 1: Fast Access

Only arithmetic operations are required.

$$A[k] \Rightarrow O(1)$$

Property 2: Fixed Element Size

Every element must occupy the same number of bytes.

$$\text{Offset} = k \times w$$

Array Representation: Deeper Observation

The address formula explains two important properties.

Property 1: Fast Access

Only arithmetic operations are required.

$$A[k] \Rightarrow O(1)$$

Property 2: Fixed Element Size

Every element must occupy the same number of bytes.

$$\text{Offset} = k \times w$$

Question

Why can a standard C array not directly store an integer, a character array and a floating-point value as independent elements?

Array Representation: Deeper Observation

The address formula explains two important properties.

Property 1: Fast Access

Only arithmetic operations are required.

$$A[k] \Rightarrow O(1)$$

Property 2: Fixed Element Size

Every element must occupy the same number of bytes.

$$\text{Offset} = k \times w$$

Question

Why can a standard C array not directly store an integer, a character array and a floating-point value as independent elements?

Two-Dimensional Arrays

A two-dimensional array represents data in rows and columns.

```
int A[3][4];
```

It contains:

$$3 \times 4 = 12$$

elements.

$$A[i][j]$$

where:

- i identifies the row.
- j identifies the column.

Two-Dimensional Arrays

A two-dimensional array represents data in rows and columns.

```
int A[3][4];
```

It contains:

$$3 \times 4 = 12$$

elements.

$$A[i][j]$$

where:

- i identifies the row.
- j identifies the column.

In C, a 2D array is conceptually:

An array of one-dimensional arrays

Logical Representation of a 2D Array

$$A = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 50 & 60 & 70 & 80 \\ 90 & 100 & 110 & 120 \end{bmatrix}$$

$$A[0][2] = 30$$

$$A[1][3] = 80$$

$$A[2][1] = 100$$

Logical Representation of a 2D Array

$$A = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 50 & 60 & 70 & 80 \\ 90 & 100 & 110 & 120 \end{bmatrix}$$

$$A[0][2] = 30$$

$$A[1][3] = 80$$

$$A[2][1] = 100$$

Question

How many complete rows are skipped before reaching $A[2][1]$?

Logical Representation of a 2D Array

$$A = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 50 & 60 & 70 & 80 \\ 90 & 100 & 110 & 120 \end{bmatrix}$$

$$A[0][2] = 30$$

$$A[1][3] = 80$$

$$A[2][1] = 100$$

Question

How many complete rows are skipped before reaching $A[2][1]$?

Two complete rows, each containing four elements.

Linear Memory vs Matrix View

A computer's memory is one-dimensional.

The matrix:

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

must be converted into a linear sequence.

Row-Major

10, 20, 30, 40, 50, 60

Column-Major

10, 40, 20, 50, 30, 60

Row-Major Address Calculation

For an array:

$$A[M][N]$$

the location of $A[i][j]$ in row-major order is:

$$LOC(A[i][j]) = BA(A) + (iN + j)w$$

For non-zero lower bounds:

$$LOC(A[i][j]) = BA(A) + [(i - L_1)N + (j - L_2)] w$$

where:

$$N = U_2 - L_2 + 1$$

Deriving the Row-Major Formula

To reach $A[i][j]$:

- 1 Skip i complete rows.
- 2 Each row contains N elements.
- 3 Skip j additional elements in the current row.

Therefore, the number of skipped elements is:

$$iN + j$$

Multiplying by the element size:

$$(iN + j)w$$

Adding the base address:

$$BA(A) + (iN + j)w$$

Numerical Example: Row-Major Order

Given:

```
int A[4][5];
```

$$BA(A) = 1000, \quad w = 4$$

Find the address of:

$A[2][3]$

Numerical Example: Row-Major Order

Given:

`int A[4][5];`

$$BA(A) = 1000, \quad w = 4$$

Find the address of:

`A[2][3]`

$$\begin{aligned} LOC(A[2][3]) &= 1000 + (2 \times 5 + 3) \times 4 \\ &= 1000 + 13 \times 4 \end{aligned}$$

1052

Column-Major Address Calculation

For an array:

$$A[M][N]$$

stored in column-major order:

$$LOC(A[i][j]) = BA(A) + (jM + i)w$$

For arbitrary lower bounds:

$$LOC(A[i][j]) = BA(A) + [(j - L_2)M + (i - L_1)] w$$

where:

$$M = U_1 - L_1 + 1$$

Row-Major vs Column-Major

For:

$$A[4][5], \quad BA = 1000, \quad w = 4$$

find $A[2][3]$.

Row-Major

$$1000 + (2 \times 5 + 3)4$$

1052

Column-Major

$$1000 + (3 \times 4 + 2)4$$

1056

Row-Major vs Column-Major

For:

$$A[4][5], \quad BA = 1000, \quad w = 4$$

find $A[2][3]$.

Row-Major

$$1000 + (2 \times 5 + 3)4$$

1052

Column-Major

$$1000 + (3 \times 4 + 2)4$$

1056

Important

C uses row-major order.

Three-Dimensional Array

Consider:

```
int A[2][3][4];
```

It contains:

$$2 \times 3 \times 4 = 24$$

elements.

It can be visualised as:

- 2 layers
- 3 rows per layer
- 4 columns per row

An element is accessed as:

$$A[i][j][k]$$

Address Formula for a 3D Array

For:

$$A[D_1][D_2][D_3]$$

stored in row-major order:

$$LOC(A[i][j][k]) = BA(A) + [iD_2D_3 + jD_3 + k] w$$

Interpretation:

- iD_2D_3 : skip complete layers.
- jD_3 : skip complete rows.
- k : move within the selected row.

Numerical Example: 3D Array

Given:

```
int A[3][4][5];
```

$$BA(A) = 2000, \quad w = 4$$

Find:

$A[2][1][3]$

Numerical Example: 3D Array

Given:

```
int A[3][4][5];
```

$$BA(A) = 2000, \quad w = 4$$

Find:

$A[2][1][3]$

$$\text{Offset} = 2(4)(5) + 1(5) + 3$$

$$= 40 + 5 + 3 = 48$$

$$LOC = 2000 + 48 \times 4$$

General N-Dimensional Mapping

For:

$$A[D_1][D_2] \cdots [D_n]$$

the row-major offset is:

$$i_1(D_2 D_3 \cdots D_n) + i_2(D_3 D_4 \cdots D_n) + \cdots + i_{n-1} D_n + i_n$$

General N-Dimensional Mapping

For:

$$A[D_1][D_2] \cdots [D_n]$$

the row-major offset is:

$$i_1(D_2 D_3 \cdots D_n) + i_2(D_3 D_4 \cdots D_n) + \cdots + i_{n-1} D_n + i_n$$

Rule

Multiply each index by the product of all dimensions to its right.

For a 4D array:

$$A[D_1][D_2][D_3][D_4]$$

$$\text{Offset} = i_1 D_2 D_3 D_4 + i_2 D_3 D_4 + i_3 D_4 + i_4$$

What is an Ordered List?

An ordered list is a collection in which elements are maintained in a specified order.

Examples:

10, 20, 35, 50, 75

or:

90, 80, 60, 40, 10

The ordering may be:

- Ascending
- Descending
- Lexicographic
- Based on a record key

What is an Ordered List?

An ordered list is a collection in which elements are maintained in a specified order.

Examples:

10, 20, 35, 50, 75

or:

90, 80, 60, 40, 10

The ordering may be:

- Ascending
- Descending
- Lexicographic
- Based on a record key

An ordered list can be implemented using an array.

Why Maintain an Ordered List?

Ordering provides important advantages:

- Minimum and maximum values are easily located.
- Binary search becomes possible.
- Duplicate detection becomes easier.
- Merging two ordered lists is efficient.
- Range-based queries become easier.

Why Maintain an Ordered List?

Ordering provides important advantages:

- Minimum and maximum values are easily located.
- Binary search becomes possible.
- Duplicate detection becomes easier.
- Merging two ordered lists is efficient.
- Range-based queries become easier.

However, maintaining order has a cost:

Insertion and deletion may require shifting

Searching an Ordered Array

Consider:

10, 20, 35, 50, 75, 90

Linear search:

$$O(n)$$

Binary search:

$$O(\log n)$$

Searching an Ordered Array

Consider:

10, 20, 35, 50, 75, 90

Linear search:

$$O(n)$$

Binary search:

$$O(\log n)$$

Key Trade-off

Ordering improves searching but makes modification more expensive.

Insertion into an Ordered List

Insert 40 into:

10, 20, 35, 50, 75

Steps:

- 1 Find the correct position.
- 2 Shift 50 and 75 one position right.
- 3 Place 40 in the empty position.

Result:

10, 20, 35, 40, 50, 75

Time complexity:

$$O(\log n) + O(n) = O(n)$$

Binary search may find the position quickly, but shifting still takes linear time.

Deletion from an Ordered List

Delete 35 from:

10, 20, 35, 40, 50, 75

Steps:

- 1 Locate 35.
- 2 Shift all following elements one position left.
- 3 Decrease the logical size.

Result:

10, 20, 40, 50, 75

Worst-case complexity:

$$O(n)$$

Ordered List Complexity

Operation	Unordered Array	Ordered Array
Access by index	$O(1)$	$O(1)$
Linear search	$O(n)$	$O(n)$
Binary search	Not directly useful	$O(\log n)$
Insert at end	$O(1)$	Depends on value
Insert in order	$O(1)$ or $O(n)$	$O(n)$
Delete known position	$O(n)$	$O(n)$
Find minimum	$O(n)$	$O(1)$

C Code: Insert into an Ordered List

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[20] = {10, 20, 35, 50, 75};
6     int n = 5, value = 40;
7     int pos = 0;
8
9     while (pos < n && a[pos] < value)
10         pos++;
11
12     for (int i = n; i > pos; i--)
13         a[i] = a[i - 1];
14
15     a[pos] = value;
16     n++;
17
18     for (int i = 0; i < n; i++)
19         printf("%d ", a[i]);
20
21     return 0;
22 }
```

What is a Sparse Matrix?

A sparse matrix is a matrix in which most elements are zero.

Example:

$$A = \begin{bmatrix} 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 2 \end{bmatrix}$$

Total elements:

$$4 \times 5 = 20$$

Non-zero elements:

$$4$$

Zero elements:

When is a Matrix Considered Sparse?

There is no single universal threshold, but a useful test is:

Number of zeros $>$ Number of non-zero elements

Sparsity may be measured as:

$$\text{Sparsity} = \frac{\text{Number of zero elements}}{\text{Total number of elements}}$$

For the previous matrix:

$$\text{Sparsity} = \frac{16}{20} = 0.8$$

80%

Why Normal Storage is Wasteful

For an $m \times n$ matrix:

$$\text{Normal storage} = mn$$

elements.

If only t elements are non-zero, storing all mn values wastes space when:

$$t \ll mn$$

Example:

$$1000 \times 1000$$

contains:

$$1,000,000$$

positions.

If only 5000 are non-zero, normal storage still reserves space for one million values

Sparse Matrix Applications

Sparse matrices occur naturally in:

- Social networks
- Road and transportation networks
- Web-page link graphs
- Recommendation systems
- Scientific simulations
- Machine-learning feature matrices
- Electrical circuit analysis
- Image processing
- Graph adjacency matrices

Sparse Matrix Applications

Sparse matrices occur naturally in:

- Social networks
- Road and transportation networks
- Web-page link graphs
- Recommendation systems
- Scientific simulations
- Machine-learning feature matrices
- Electrical circuit analysis
- Image processing
- Graph adjacency matrices

Example

In a social network, one user is connected to only a small fraction of all users. Therefore, the adjacency matrix contains mostly zeros.

Triplet Representation

Instead of storing every zero, store only:

(Row, Column, Value)

For:

$$A = \begin{bmatrix} 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 2 \end{bmatrix}$$

the non-zero entries are:

(0, 3, 9)

(1, 1, 8)

Three-Tuple or Triplet Table

A common representation stores matrix metadata in the first row.

Row	Column	Value
4	5	4
0	3	9
1	1	8
3	0	5
3	4	2

The first row means:

4 rows, 5 columns, 4 non-zero elements
--

The remaining rows store the non-zero entries.

Storage Comparison

Suppose an $m \times n$ matrix has t non-zero values.

Normal Representation

$$mn$$

values are stored.

Triplet Representation

$$3(t + 1)$$

values are stored.

Triplet representation is beneficial when:

$$3(t + 1) < mn$$

Storage Comparison

Suppose an $m \times n$ matrix has t non-zero values.

Normal Representation

$$mn$$

values are stored.

Triplet Representation

$$3(t + 1)$$

values are stored.

Triplet representation is beneficial when:

$$3(t + 1) < mn$$

For a 100×100 matrix with 100 non-zero values:

Numerical Question: Storage Efficiency

A 20×30 matrix contains 25 non-zero values.

Normal storage:

$$20 \times 30 = 600$$

Triplet storage:

$$3(25 + 1) = 78$$

Percentage reduction:

$$\begin{aligned} \frac{600 - 78}{600} \times 100 \\ = 87\% \end{aligned}$$

Approximately 87% fewer integer locations

C Structure for a Sparse Matrix Entry

A non-zero element may be represented using a structure.

```
1 typedef struct
2 {
3     int row;
4     int column;
5     int value;
6 } Term;
```

An array of structures can store all non-zero terms:

```
1 Term sparse[MAX_TERMS];
```

Advantages:

- Clear field names
- Easy traversal
- Easy transpose and addition
- Avoids storing zero values

C Code: Convert Matrix to Triplet Form

```
1 #include <stdio.h>
2
3 #define ROWS 4
4 #define COLS 5
5
6 int main()
7 {
8     int matrix[ROWS][COLS] =
9     {
10         {0, 0, 0, 9, 0},
11         {0, 8, 0, 0, 0},
12         {0, 0, 0, 0, 0},
13         {5, 0, 0, 0, 2}
14     };
15
16     int triplet[ROWS * COLS + 1][3];
17     int k = 1;
18
19     for (int i = 0; i < ROWS; i++)
20     {
21         for (int j = 0; j < COLS; j++)
22         {
```

Output of the Triplet Program

Row	Column	Value
4	5	4
0	3	9
1	1	8
3	0	5
3	4	2

Complexity

Every element of the original matrix is examined once.

$$O(mn)$$

The final sparse structure occupies:

$$O(t)$$

Special Sparse Matrices

Some structured matrices can be stored even more efficiently.

- Diagonal matrix
- Lower triangular matrix
- Upper triangular matrix
- Symmetric matrix
- Tri-diagonal matrix

Special Sparse Matrices

Some structured matrices can be stored even more efficiently.

- Diagonal matrix
- Lower triangular matrix
- Upper triangular matrix
- Symmetric matrix
- Tri-diagonal matrix

A tri-diagonal matrix has non-zero elements only:

- On the main diagonal
- Immediately above the main diagonal
- Immediately below the main diagonal

$$A[i][j] = 0 \quad \text{when} \quad |i - j| > 1$$

Tri-Diagonal Matrix Storage

For an $n \times n$ tri-diagonal matrix:

- Main diagonal contains n elements.
- Upper diagonal contains $n - 1$ elements.
- Lower diagonal contains $n - 1$ elements.

Total potentially non-zero elements:

$$n + (n - 1) + (n - 1)$$

$$3n - 2$$

Instead of storing:

$$n^2$$

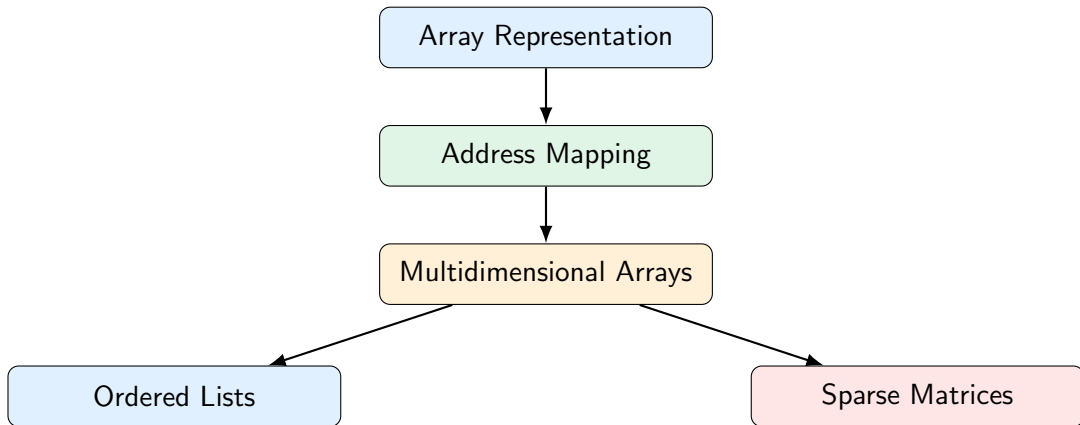
elements, only:

$$3n - 2$$

Deep Comparison

Structure	Main Benefit	Main Cost	Typical Access	Access
Ordinary array	Direct indexing	Fixed size	$O(1)$	
Ordered array	Binary search	Costly insertion	$O(\log n)$ search	
2D array	Natural matrix view	Stores every position	$O(1)$	
Sparse triplet	Memory saving	Indirect element lookup	Up to $O(t)$	
Tri-diagonal array	Very compact storage	Only supports special form	$O(1)$ with mapping	

Concept Integration



Rapid-Fire Questions

- 1 Why is access to $A[i]$ constant time?

Rapid-Fire Questions

- 1 Why is access to $A[i]$ constant time?
- 2 Which dimension is multiplied in the row-major formula?

Rapid-Fire Questions

- 1 Why is access to $A[i]$ constant time?
- 2 Which dimension is multiplied in the row-major formula?
- 3 Can binary search be applied directly to an unordered list?

Rapid-Fire Questions

- ① Why is access to $A[i]$ constant time?
- ② Which dimension is multiplied in the row-major formula?
- ③ Can binary search be applied directly to an unordered list?
- ④ Why is insertion still $O(n)$ in an ordered array?

Rapid-Fire Questions

- 1 Why is access to $A[i]$ constant time?
- 2 Which dimension is multiplied in the row-major formula?
- 3 Can binary search be applied directly to an unordered list?
- 4 Why is insertion still $O(n)$ in an ordered array?
- 5 What information does one sparse triplet store?

Rapid-Fire Questions

- 1 Why is access to $A[i]$ constant time?
- 2 Which dimension is multiplied in the row-major formula?
- 3 Can binary search be applied directly to an unordered list?
- 4 Why is insertion still $O(n)$ in an ordered array?
- 5 What information does one sparse triplet store?
- 6 When is triplet representation memory efficient?

Tutorial Problems

- 1 An array $A[-5 \dots 10]$ has base address 1000 and element size 4 bytes. Find $LOC(A[7])$.
- 2 A matrix $B[4][6]$ has base address 2000 and stores 2-byte integers. Find $LOC(B[3][4])$ in row-major order.
- 3 Find the address of $B[3][4]$ in column-major order.
- 4 Insert 42 into the ordered list:

10, 18, 30, 50, 65

- 5 A 50×60 matrix has 100 non-zero elements. Compare normal storage with triplet storage.
- 6 How many values are sufficient to store an $n \times n$ tri-diagonal matrix?

Tutorial Answers

1

$$1000 + (7 + 5)4 = \boxed{1048}$$

2

$$2000 + (3 \times 6 + 4)2 = \boxed{2044}$$

3

$$2000 + (4 \times 4 + 3)2 = \boxed{2038}$$

4

$$10, 18, 30, \boxed{42}, 50, 65$$

5

$$3000 \text{ versus } 3(101) = 303$$

6

$$\boxed{3n - 2}$$

Exit Problem

Given:

$$A[2 \dots 5][-3 \dots 4]$$

$$BA(A) = 1000, \quad w = 4$$

Find the address of:

$$A[4][2]$$

assuming row-major order.

Exit Problem

Given:

$$A[2 \dots 5][-3 \dots 4]$$

$$BA(A) = 1000, \quad w = 4$$

Find the address of:

$$A[4][2]$$

assuming row-major order.

Number of columns:

$$N = 4 - (-3) + 1 = 8$$

$$LOC(A[4][2]) = 1000 + [(4 - 2)8 + (2 - (-3))]4$$

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Computer Science Press.
- ② Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
- ③ Reema Thareja, *Data Structures Using C*, Oxford University Press.

Questions?

Thank You